

Mohsin Iban Hossain

AIUB, AI Final Term Notes

ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEM

Table of Contents		
Sl. No	Topic	Pages
1	Local Search Algorithms	
2	Hill-Climbing Algorithms	
3	Simulated Annealing	
4	Local Beam Search	
5	Genetic Algorithms	
6	Alpha-Beta Pruning Algorithm	
7	Constraint Satisfaction Problems (CSPs)	
8	Bayesian network	
9		
10		
11		
12		
13		

LOCAL SEARCH ALGORITHMS

WHAT IS LOCAL SEARCH?

- A search strategy that **works with a single current state** and moves to **neighboring states**.
- Unlike BFS/DFS, it **does not build a full path tree**.
- Useful for **optimization problems** where the goal state is not known, but the objective is to **maximize/minimize a function**.

Characteristics

- Works with **complete states**, not paths.
- Uses an **evaluation (objective/fitness) function** instead of path cost.
- Requires **little memory** ($O(1)$ or $O(\log n)$).
- May get stuck in **local optima** (not global best solution).

Common Local Search Algorithms

a) Hill-Climbing (Greedy Local Search):

- Always move to the neighbor with the **best improvement** in the evaluation function.
- Stops when no neighbor is better → can get stuck in:
 - **Local Maxima**
 - **Plateaus** (flat area with equal values)
 - **Ridges** (hard to climb directly)

➡ Variants:

- **Steepest-Ascent Hill Climbing** (choose best among all neighbors).
- **Stochastic Hill Climbing** (pick a random better neighbor).

b) Simulated Annealing:

- Inspired by **annealing in metallurgy** (heating and cooling metal).
- Randomly choose a neighbor:
 - If it's better → accept it.
 - If it's worse → accept it with **some probability** (to escape local maxima).
- The probability of accepting worse moves decreases over time (temperature cooling).

c) Local Beam Search:

- Keep **k states** instead of one.
- At each step:
 - Generate neighbors of all **k** states.
 - Choose the **best k** among them.
- Helps avoid being stuck in a poor local maximum by exploring multiple candidates.

d) Genetic Algorithms (GA) (also considered local search):

- Based on **biological evolution**.
- Population of candidate solutions → evolve using:
 - **Selection** (best survive)
 - **Crossover** (combine solutions)
 - **Mutation** (small random changes)

Applications of Local Search

- **Optimization problems** (e.g., traveling salesman, scheduling, circuit layout).
- **Games** (chess evaluation, strategy optimization).
- **Machine learning** (neural network weight optimization).
- **Robotics** (path planning with constraints).

HILL-CLIMBING ALGORITHMS

Hill-Climbing (Greedy Local Search)

- ⇒ **Hill-Climbing** is a **local search algorithm** that continuously moves to the neighbor with the **highest value** (or steepest uphill move) in the search space.
- ⇒ It's called "hill climbing" because the algorithm metaphorically climbs to the top of a hill (local maximum) using **greedy steps**.

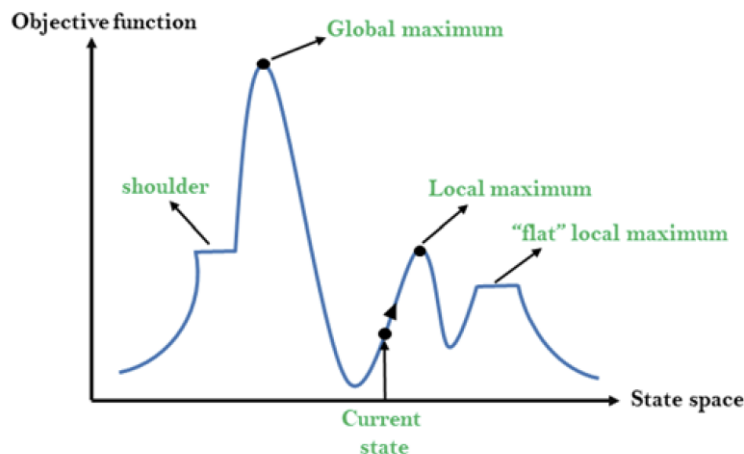
Note: It **ignores paths** and only looks at the **current state** and its **neighbors**.

Algorithm Steps

1. Start at an initial state.
2. Evaluate neighbors using the **heuristic/fitness function**.
3. If one neighbor is **better**, move to that neighbor.
4. Repeat until **no better neighbor exists** (i.e., a peak is reached).

State-space Diagram for Hill Climbing

- ⇒ The **state-space landscape** is a **graph used** to explain the **hill-climbing algorithm**.
 - **X-axis** → states of the algorithm
 - **Y-axis** → objective function (maximize) or cost function (minimize)
- If Y-axis = **cost**, the goal is to reach the **lowest point** (global or local minimum).
- If Y-axis = **objective function**, the goal is to reach the **highest point** (global or local maximum).

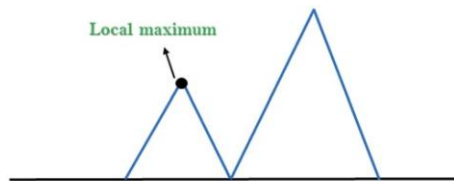


Different regions in the state space landscape

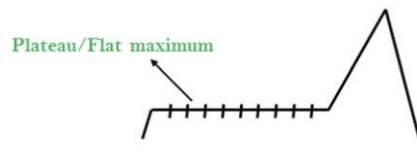
- ⇒ **Local Maximum** → A peak that is higher than nearby states, but not the highest overall.
- ⇒ **Global Maximum** → The highest peak in the whole landscape (best state).
- ⇒ **Current State** → The position where the agent is right now.
- ⇒ **Flat Local Maximum** → A flat region where all nearby states have the same value.
- ⇒ **Shoulder** → A flat plateau that still has an uphill path leading higher.

Problems in Hill Climbing Algorithm

1. **Local Maximum:** A peak that is higher than its neighbors, but not the highest overall.



2. **Plateau:** A flat region in the search space where all nearby states have the same value. The algorithm gets stuck because it can't find a better direction to move.

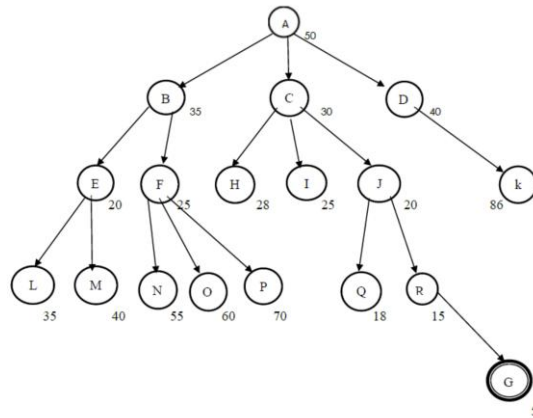


3. **Ridges:** A narrow, sloping peak that is higher than the surroundings but hard to reach in one step.



✂ Problem1:

>> Using the given search tree, find the path from the start node **A** to the goal node **G** by always choosing the **minimum cost value** at each step. Show the step-by-step selection of nodes until the goal is reached.



⇒ Solutions:

Iterations	Open	Close	Visit
1	Open [A_{50}]	Close []	NULL
2	Open [B_{35}, C_{30}, D_{40}]	Close [A]	A
3	Open [H_{28}, I_{25}, J_{20}]	Close [A, C]	C
4	Open [Q_{18}, R_{15}]	Close [A, C, J]	J
5	Open [G_5]	Close [A, C, J, R]	R
6	Open []	Close [A, C, J, R, G]	G (GOAL)
Solution Path: A → C → J → R → G			

Properties of Hill-Climbing

Property	Description
Complete	No (can get stuck in local maxima/plateau)
Optimal	No
Time	Depends on number of neighbors and steps
Space	O(1) – very efficient

**SIMULATED
ANNEALING**

Simulated Annealing

- ⇒ **Simulated Annealing (SA)** is a **probabilistic local search algorithm** inspired by the physical process of **annealing in metallurgy**.
- ⇒ In metallurgy, a material is **heated** and then slowly **cooled** so atoms can settle into a low-energy stable structure.
- ⇒ In AI, SA uses a similar process to escape **local maxima** by **occasionally accepting worse moves** with some probability.

Note: **Hill-Climbing:** always step uphill, can get stuck on a small hill. **Simulated Annealing:** sometimes step downhill (with decreasing probability), which allows escaping small hills and possibly reaching the **highest peak**.

Algorithm Steps

1. Start at an initial state with an initial **temperature T**.
2. At each step:
 - a. Pick a **random neighbor**.
 - b. Compute the change in value: $\Delta E = \text{Value}(\text{neighbor}) - \text{Value}(\text{current})$.
 - c. If $\Delta E > 0 \rightarrow$ accept the move (better state).
 - d. If $\Delta E \leq 0 \rightarrow$ accept the move with probability:

$$P = e^{\frac{\Delta E}{T}}$$

(This allows moving to a worse state).

3. Gradually **decrease T** (cooling schedule).
4. Stop when system is "frozen" ($T \approx 0$) or goal reached.

❖ Why Simulated Annealing Works:

- ⇒ In early stages (high T), the algorithm is **more random** → explores widely.
- ⇒ As T decreases, the algorithm becomes **more greedy** → converges to a solution.
- ⇒ Occasional acceptance of worse moves prevents getting stuck at **local maxima**.

Properties of Simulated Annealing

Property	Description
Complete	No (randomized, might fail to find goal)
Optimal	Yes (in theory, if cooling schedule $\rightarrow \infty$ slowly)
Time	Depends on cooling schedule
Space	$O(1)$ – very efficient

LOCAL BEAM SEARCH

Local Beam Search

- ⇒ **Local Beam Search** is a **local search algorithm** that keeps track of **k different states (candidates)** instead of just one (like Hill-Climbing).
- ⇒ At each step:
 - It generates all successors of these **k** states.
 - Chooses the **best k successors** to form the new set of states.
- ⇒ This process continues until a solution is found.

Note: By keeping multiple candidates, the algorithm reduces the risk of getting stuck in **local maxima**.

Algorithm Steps

1. Generate **k** random initial states.
2. Repeat until a solution is found:
 - For each state, generate all its neighbors.
 - From all generated neighbors, choose the **best k**.
 - Replace the old set with the new set.

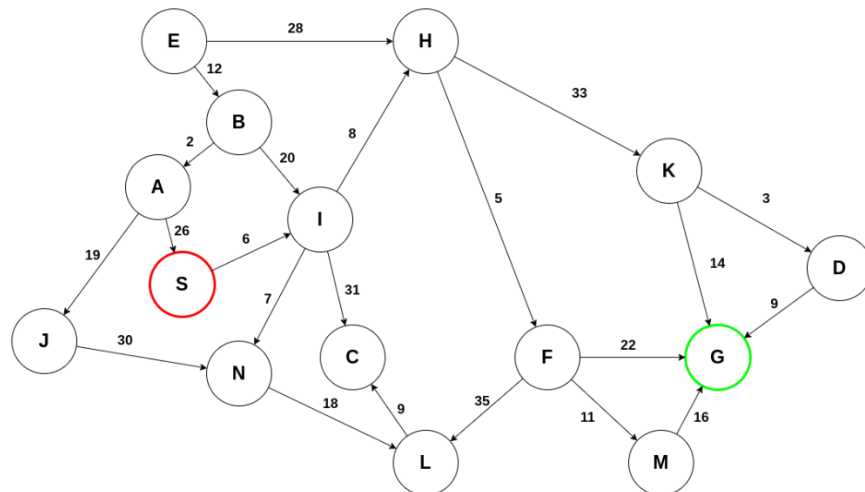
Properties of Local Beam Search

Property	Value
Complete	No (may still get stuck in local maxima)
Optimal	No
Time	$O(k * b^m)$ (depends on k and neighbors)
Space	$O(k)$ (stores only k states, not the whole tree)

✂ Problem1:

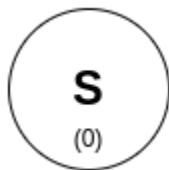
>> Consider the following graph where the numbers represent evaluation values for each node. The goal is to search from **S** (start) to **G** (goal) using **Beam Search**. Assume the **beam width $\beta = 2$** , meaning at each level only the best two nodes (with the lowest values) are kept for further expansion.

1. Show step-by-step how the beam search progresses from the start node **S** until it reaches the goal node **G**.
2. Write down the final path found by the algorithm.



⇒ **Solutions:**

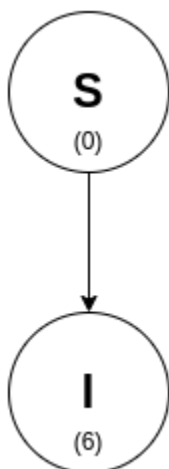
Exploring S: Initialize the tree and add S to the closed list:



Node[cost]
I[6]

Closed List
S

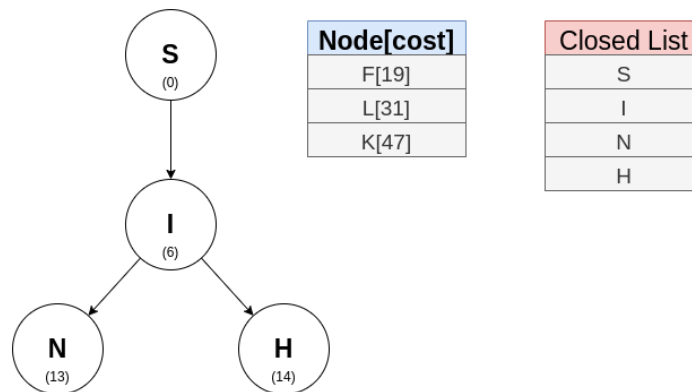
S only has one adjacent node, so it is added to the tree:



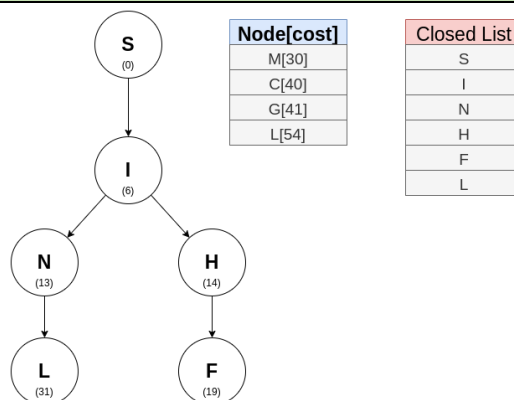
Node[cost]
N[13]
H[14]
C[37]

Closed List
S
I

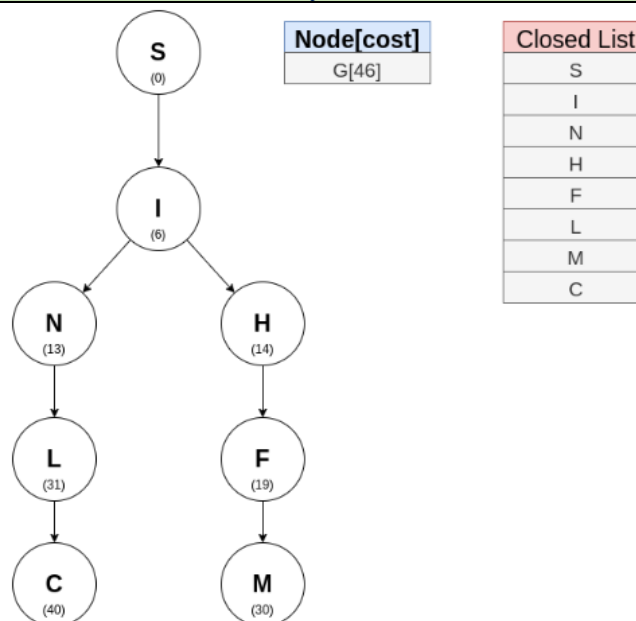
Explore **I**. Nodes **N** and **H** are the top two nodes, so those are added to the tree:



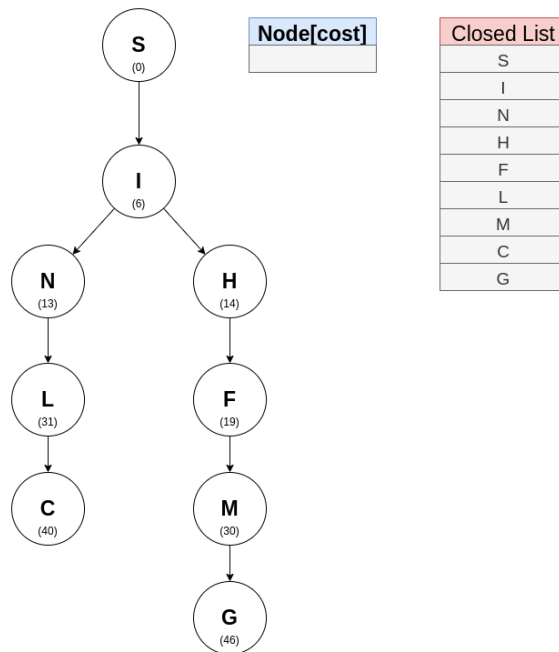
Nodes **F** (adjacent to **H** in the graph) and **L** (adjacent to **N** in the graph) are next:



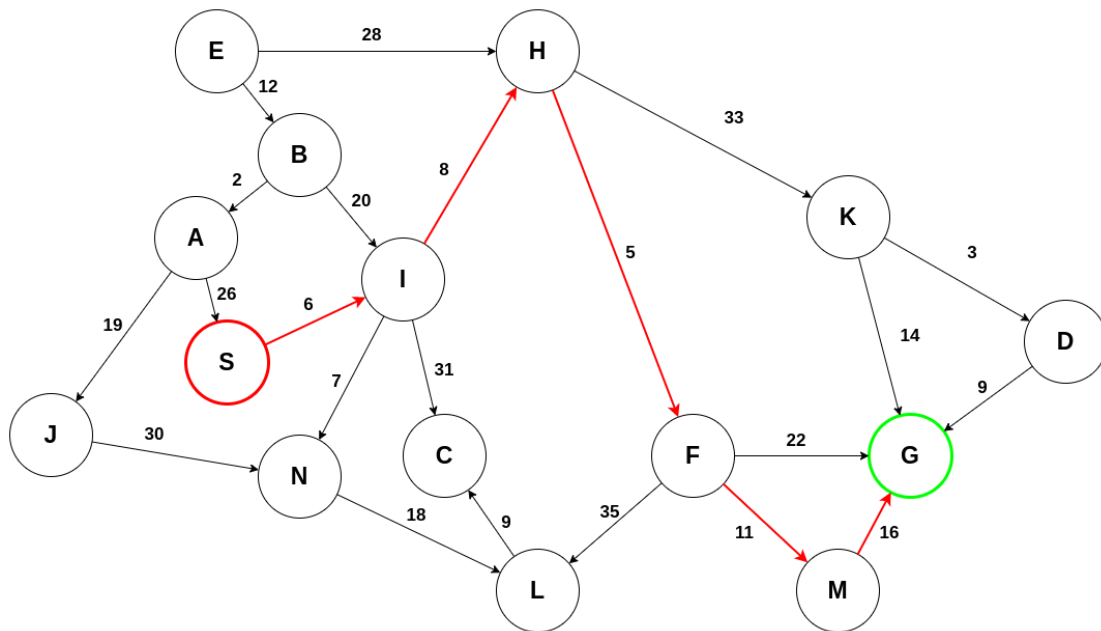
The incompleteness of this algorithm can be seen in this step. Node **G** has appeared in the **open list** but unfortunately, it has been eliminated by the beam width. Nodes **M** and **C** are next:



It is in this step that node **G** has been found:



Trace the path from **S** to **G**:



Solution Path: **S** → **I** → **H** → **F** → **M** → **G**

Solution Path Cost: **46**

GENETIC ALGORITHMS

Genetic Algorithms

- ⇒ A **Genetic Algorithm (GA)** is a **search and optimization technique** inspired by the process of **natural selection** and **evolution**.
- ⇒ Instead of working on a single solution, GA works with a **population of solutions** and evolves them over time.
- ⇒ Uses **selection, crossover, and mutation** to create new solutions.

Idea: “*Survival of the fittest*” - good solutions survive and combine to form even better solutions.

Biological Background

- **Chromosome:** Representation of a candidate solution (e.g., binary string, array of values).
- **Gene:** A part of a chromosome (represents one decision variable).
- **Population:** Set of all candidate solutions.
- **Fitness Function:** Measures how good a solution is.
- **Generation:** One cycle of evolution (selection → crossover → mutation).

Working of Genetic Algorithms

1. Initialization

- Create a random population of candidate solutions.

2. Evaluation

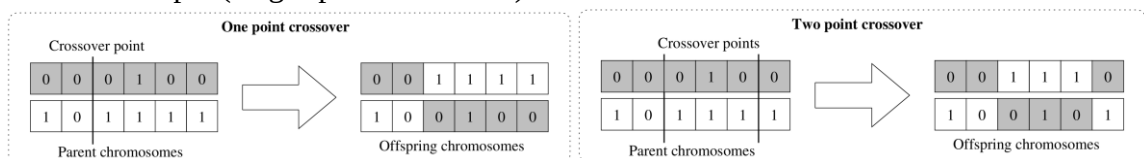
- Compute the **fitness** of each candidate using the fitness function.

3. Selection

- Select parents based on fitness (fitter individuals have higher chances).

4. Crossover (Recombination)

- Combine two parents to produce offspring.
- Example (single-point crossover):



5. Mutation

- Randomly flip or alter some genes to introduce diversity.
- Example:

Before: 110011 → After: 111011

6. Replacement

- Form a new population (offspring replace weaker individuals).

7. Termination

- Repeat until a stopping condition is met (max generations or good enough fitness).

PSEUDOCODE

```
GeneticAlgorithm():
    population ← initialize_random()
    while not termination_condition:
        fitness ← evaluate(population)
        parents ← select(population, fitness)
        offspring ← crossover(parents)
        offspring ← mutate(offspring)
        population ← select_new_population(population, offspring)
    return best_solution(population)
```

PROPERTIES

Property	Description
Complete	No (stochastic, may not find solution)
Optimal	No (but often finds near-optimal solutions)
Time	Depends on population size × generations
Space	O(population size)

ENCODING

⇒ **Encoding** defines how a candidate solution (chromosome) is represented.

➤ **Common methods:**

1. Binary Encoding (most common)
2. Permutation Encoding
3. Value Encoding
4. Tree Encoding

Binary Encoding

⇒ In **Binary Encoding**, each solution (**chromosome**) is represented as a string of **0s and 1s**.

Example:

Chromosome 1: **10110011**

Chromosome 2: **10111011**

Permutation Encoding

- ⇒ **Permutation Encoding**, a solution (**chromosome**) is represented as a **sequence (permutation)** of numbers.
- ⇒ Each number represents a **position, order, or object**, and no number is repeated.

Example:

[3, 5, 1, 4, 2]

Visit city 3 → then 5 → then 1 → then 4 → then 2.

Value Encoding

- ⇒ In **Value Encoding**, a solution (**chromosome**) is represented as a string of **real values, characters, or symbols** (instead of binary digits or permutations).

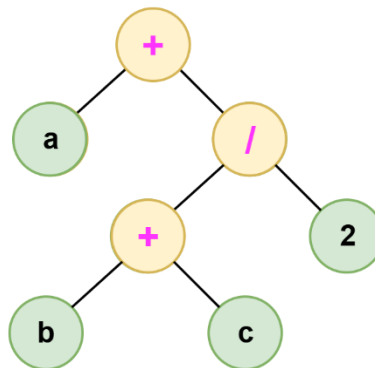
Example:

[3.14, 7.2, 1.5, 9.8]

Tree Encoding

- ⇒ In **Tree Encoding**, each solution (**chromosome**) is represented as a **tree structure**.

Example:



This represents the mathematical expression:

$a + (b + c) / 2$

Operators of GA

1. Selection (Survival of the Fittest)
2. Crossover (Recombination)
3. Mutation

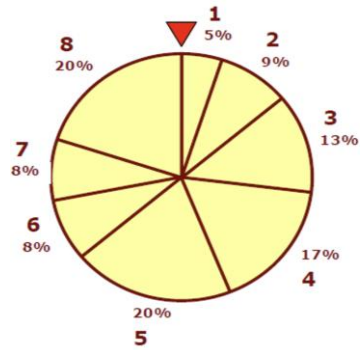
Selection

⇒ **Selection** is the process of choosing individuals (chromosomes) from the **current population** to create the **next generation**.

Selection Methods:

1. Roulette Wheel Selection (Fitness Proportionate Selection)

- Each individual gets a slot on the “roulette wheel” proportional to its fitness.
- Spin the wheel → individuals with **higher fitness** have **higher probability** of being chosen.

Example:	
	Population fitness values:
	[5, 9, 13, 17, 20, 8, 8, 20]
	Probabilities:
	[0.05, 0.09, 0.13, 0.17, 0.20, 0.08, 0.08, 0.20]

2. Tournament Selection:

- Randomly choose a subset of individuals.
- The **best among them** is selected.

Example: Pick 3 random candidates → select the one with highest fitness.

3. Rank Selection

- Individuals are sorted by fitness rank.
- Selection probability is based on **rank**, not actual fitness value.
- Prevents very strong individuals from dominating too quickly.

4. Elitism (Special Case)

- The **best individuals** are always carried forward to the next generation without change.
- Guarantees that the **best solution so far** is never lost.

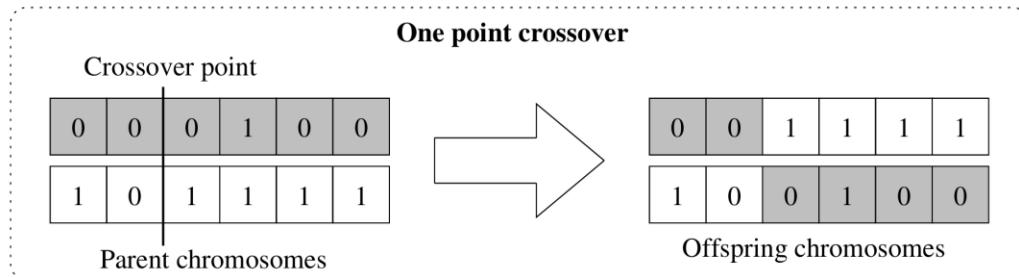
Crossover

⇒ **Crossover (Recombination)** is a GA operator used to combine the **genetic information** of two parent chromosomes to produce **new offspring**.

Types of Crossover:

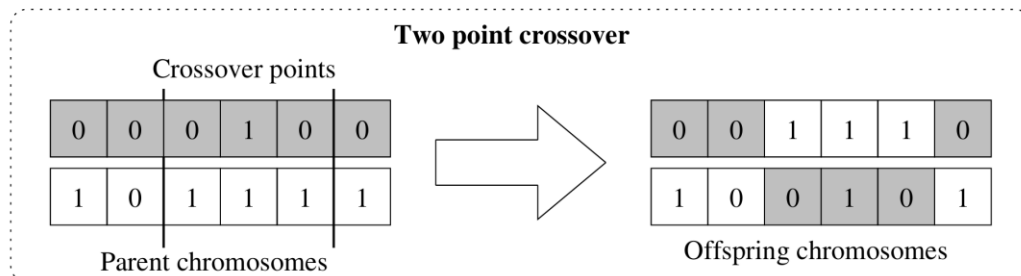
1. Single-Point Crossover

⇒ A random point is chosen, and chromosomes exchange segments.



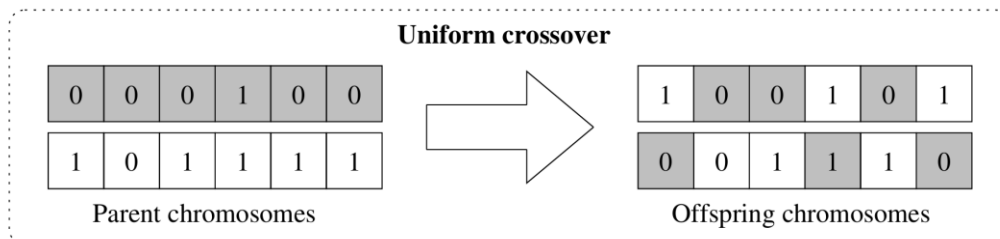
2. Two-Point Crossover

⇒ Two points are chosen, and the middle segment is swapped.



3. Uniform Crossover

⇒ Each gene is chosen randomly from either parent with equal probability.



Example: Uniform Crossover							
Parent P ₁	A	B	C	D	E	F	G
Parent P ₂	E	B	D	C	F	G	A
Template	0	1	1	0	0	1	0
Child C ₁	E	B	C	D	G	F	A
Child C ₂	A	B	D	C	E	G	F

Figure 4: Illustration of uniform order crossover.

4. Arithmetic / Blend Crossover (for real values)

⇒ For numerical genes, offspring are created using linear combinations of parents.

Formula:	Example: $\alpha = 0.5 \rightarrow$ average
$\text{Offspring1} = \alpha \times \text{Parent1} + (1 - \alpha) \times \text{Parent2}$ $\text{Offspring2} = (1 - \alpha) \times \text{Parent1} + \alpha \times \text{Parent2}$ Where $0 < \alpha < 1$	Parent1 = [2.5, 4.0] Parent2 = [3.5, 7.0] Child1 = [3.0, 5.5] Child2 = [3.0, 5.5]

5. Heuristic Crossover

⇒ **Heuristic Crossover** is a **knowledge-guided crossover operator** used in GAs.

⇒ Unlike standard crossover (which randomly exchanges genes), it uses a **heuristic rule** to decide how offspring are created.

Working Principle

- Suppose we have two parents:
 - Parent A** (better fitness)
 - Parent B** (worse fitness)
- The offspring is generated using the formula:

$$\text{Offspring} = \text{Parent A} + r \times (\text{Parent A} - \text{Parent B})$$

where r is a random number in $[0,1]$

Example:	
⇒ Parent A = [5, 8] (fitness = good) ⇒ Parent B = [2, 6] (fitness = poor) ⇒ Random $r = 0.5$	$\begin{aligned} \text{Offspring} &= [5, 8] + 0.5 \times ([5, 8] - [2, 6]) \\ &= [5, 8] + 0.5 \times ([3, 2]) \\ &= [5, 8] + [1.5, 1] = [6.5, 9] \end{aligned}$

Mutation

- ⇒ Mutation **randomly alters one or more genes** in a chromosome to maintain diversity and explore new solutions.
- ⇒ Unlike crossover (which combines parent genes), mutation randomly alters one or more genes in a chromosome.

Note: Introduce small random changes to individuals to explore new regions of the solution space and prevent premature convergence.

Example:											
Suppose we have a chromosome: Chromosome: [110010001010]											
Before Mutations:											
1	1	0	0	1	0	0	0	1	0	1	0
After Mutations:											
0	1	0	0	0	0	0	0	0	0	1	0

✂ Problem1:

» A **Genetic Algorithm (GA)** is used to solve an optimization problem. Each candidate solution is represented as a **6-bit binary chromosome**. The **fitness function** is defined as the **number of 1s in the chromosome** (the more 1s, the better the fitness).

The initial population is given as:

P1: 110010
P2: 101111
P3: 011001
P4: 000111

The GA applies the following operators in one generation:

1. **Selection:** Select the two fittest chromosomes as parents.
2. **Crossover:** Apply **single-point crossover** after the 3rd bit to generate two offspring.
3. **Mutation:** Flip one random bit in one of the offspring.
4. **Replacement:** Form the new population by replacing the two weakest chromosomes with the new offspring.

Tasks:

- a) Calculate the fitness of all chromosomes in the initial population.
- b) Identify the two parents selected for reproduction.
- c) Perform the crossover operation and show the offspring.
- d) Apply mutation on one offspring and show the result.
- e) Write the final new population after this generation.

→ Solutions:

a) Fitness of all initial chromosomes • P1 = 110010 → ones = 3 • P2 = 101111 → ones = 5 • P3 = 011001 → ones = 3 • P4 = 000111 → ones = 3	b) Parents selected for reproduction <u>Select the two fittest:</u> • Best: P2 (fitness 5). • Next best: a three-way tie between P1, P3, P4 (all fitness 3). By tiebreaker choose the earliest listed → P1 (fitness 3). <u>Selected parents:</u> ○ P2 = 101111 ○ P1 = 110010.
c) Crossover (single-point after 3rd bit) <u>Split parents after bit 3:</u> • P2 = 101 111 • P1 = 110 010 <u>Offspring produced:</u> • Offspring1 = left(P2) + right(P1) = 101010 • Offspring2 = left(P1) + right(P2) = 110111 <u>Fitness before mutation:</u> • Offspring1 101010 → ones = 3 • Offspring2 110111 → ones = 5	d) Mutation (flip one bit in one offspring) <u>Mutate Offspring1: flip bit 4:</u> • Offspring1 before: 101 010 = 101010 • Flip bit 4 (0 → 1) → 101 110 = 101110 <u>Fitness after mutation:</u> • Mutated Offspring1 101110 → ones = 4 • Offspring2 110111 → ones = 5 (unchanged)
e) New population after replacement Weakest chromosomes in initial population (fitness 3): P3 and P4. Replace them with the two offspring. <u>New population:</u> P2 = 101111 (fitness 5) — kept parent P1 = 110010 (fitness 3) — kept parent O1* = 101110 (fitness 4) — offspring1 after mutation O2 = 110111 (fitness 5) — offspring2 Fitnesses in new population: 5, 3, 4, 5. Best chromosomes now: 101111 and 110111 (both fitness = 5).	

✂ Problem2:

>> Given the following chromosomes (12 bits each):

C1 = 110010101111

C2 = 101101001010

C3 = 111000111000

- Apply **single-point crossover** between C1 and C2 after the 6th bit.
- Apply **two-point crossover** between C2 and C3 after the 4th and 8th bits.
- Perform **mutation** on the first offspring (from part 1). If $r < p$, flip the bit. Use mutation probability $p = 1/12 \approx 0.0833$ with random numbers:

$r = [0.04, 0.51, 0.12, 0.93, 0.02, 0.88, 0.33, 0.75, 0.07, 0.95, 0.21, 0.62]$

➔ Solutions:

a) Single-point crossover (C1 × C2 after 6th bit):						b) Two-point crossover (C2 × C3 after 4th & 8th bits):					
C1 = 110010 101111 C2 = 101101 001010 O1 = 110010001010 O2 = 101101101111						C2 = 1011 0100 1010 C3 = 1110 0011 1000 O3 = 101100111010 O4 = 111001001000					
c) Mutation on O1 (110010001010):											
Before Mutations:											
1	1	0	0	1	0	0	0	1	0	1	0
0.04	0.51	0.12	0.93	0.02	0.88	0.33	0.33	0.07	0.95	0.21	0.62
After Mutations:											
0	1	0	0	0	0	0	0	0	0	1	0

✂ **Problem3:** Consider a big businessman who has **4 warehouses** and he supplies his goods to **4 markets**. The distance in KM of each market from each warehouse is given in the table below. Each warehouse supplies goods to one market i.e., demand of each market is satisfied by one warehouse.

	Market-1	Market-2	Market-3	Market-4
WH-1	23	27	19	23
WH-2	15	10	14	8
WH-3	11	14	25	32
WH-4	9	22	13	8

Tasks:

- Generate 4 initial populations.
- Apply crossover between two chromosomes using random number thresholding.
- Apply crossover again, then find the result of iteration-3.
- Comment on the result of iteration-2 and 3.

Let, Distance matrix (rows = WH-1...WH-4, cols = Market-1...Market-4):

23	27	19	23
15	10	14	08
11	14	25	32
09	22	13	18

And, Fitness used for reporting: **fitness** = $1 / (1 + \text{total_cost})$ (higher is better).

(i) Generate 4 initial chromosomes

let's picked four valid permutations:

- A = [1, 2, 3, 4] → cost = 23 + 10 + 25 + 8 = **66**, fitness ≈ **0.01493**
- B = [2, 3, 4, 1] → cost = 15 + 14 + 13 + 23 = **65**, fitness ≈ **0.01515**
- C = [3, 4, 1, 2] → cost = 11 + 22 + 19 + 8 = **60**, fitness ≈ **0.01639**
- D = [4, 1, 2, 3] → cost = 9 + 27 + 14 + 32 = **82**, fitness ≈ **0.01205**

Best in initial population: C (cost 60).

(ii) Apply crossover between two chromosomes (random thresholding)

We have:

- ⇒ B = [2, 3, 4, 1]
- ⇒ C = [3, 4, 1, 2]
- ⇒ Threshold (th) = 0.5
- ⇒ Randoms = [0.32, 0.61, 0.27, 0.83]

Rule:

- For **Child-1**: take gene from **C** if **rand** < **th**, else from **B**.
- For **Child-2**: take gene from **B** if **rand** < **th**, else from **C**.

Offspring produced:

- Offspring1 = 3,3,1,1 = 3,2,1,4 (after remove duplicate) Cost = 48, fitness ≈ 0.02041
- Offspring2 = 2,4,4,2 = 2,4,1,3 (after remove duplicate) Cost = 88, fitness ≈ 0.01124

(iii) Apply crossover again (iteration-3)

We have:

- ⇒ C = [3, 4, 1, 2]
- ⇒ O1 = [3, 2, 1, 4]
- ⇒ Threshold (th) = 0.5
- ⇒ Randoms = [0.12, 0.77, 0.44, 0.68]

Offspring produced:

- Offspring1 = 3,4,1,2 Cost = 11+22+19+08 = 60, fitness ≈ 0.01639
- Offspring2 = 3,2,1,4 Cost = 11+10+19+18 = 58, fitness ≈ 0.02041 (NEW BEST)

(iv) Comments on iteration-2 and 3

- After the first crossover (iteration-2) the GA discovered the best solution $[3, 2, 1, 4]$ with total distance **48**.
- The second crossover (iteration-3) did not improve further; the GA **converged** and preserved the optimum. This indicates the algorithm reached the global best assignment for the given distance matrix.

✂ **Problem4:** Find the fitness value (Non attacking pair) of the following populations [21134, 40323, 20124] for the 5-Queens problem.

Fitness Evaluation

The fitness function is based on the number of non-attacking queen pairs (higher = better).

$$\text{Fitness} = F1 + F2 + F3 + F4 + F5$$

where **Fi** counts non-attacking pairs involving Qi (without double counting).

Chromosomes		
2 1 1 3 4	4 0 3 2 3	2 0 1 2 4
<i>Fitness = 3 + 1 + 1 + 0 + 0 = 5</i>	<i>Fitness = 4 + 1 + 0 + 0 + 0 = 5</i>	<i>Fitness = 3 + 1 + 1 + 1 + 0 = 6</i>

OR

Total possible queen pairs

If there are **N queens**, the number of possible pairs is:

$$\text{TotalPairs} = \frac{N(N-1)}{2}$$

Fitness value (non-attacking pairs)

We need to **count** pairs that **attack** each other.:

$$\text{Fitness} = \text{TotalPairs} - \text{AttackingPairs}$$

Chromosomes		
2 1 1 3 4	4 0 3 2 3	2 0 1 2 4
<i>Fitness</i> = 10 – 5 = 5	<i>Fitness</i> = 10 – 5 = 5	<i>Fitness</i> = 10 – 4 = 6

ALPHA-BETA PRUNING ALGORITHM

Alpha-Beta Pruning Algorithm

- ⇒ **Alpha-Beta Pruning** is an optimization technique for the **Minimax Algorithm** used in **adversarial search** (two-player games like chess, tic-tac-toe).
- ⇒ It reduces the number of nodes evaluated in the search tree by **eliminating branches** that cannot possibly affect the final decision.

Idea: “Don’t explore moves that will never be chosen by rational players.”

Key Concepts

- **Alpha (α):** Best (maximum) value that the maximizer can guarantee so far.
- **Beta (β):** Best (minimum) value that the minimizer can guarantee so far.

➤ While traversing the tree:

- **Maximizer** updates α (wants higher values).
- **Minimizer** updates β (wants lower values).

If $\alpha \geq \beta$, the branch is **cut off** (no need to explore further). This is called **pruning**.

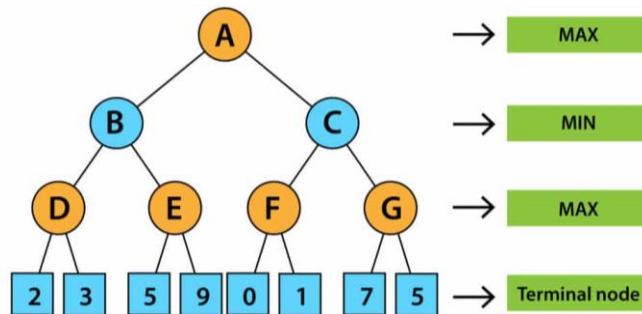
Algorithm Steps

1. initialize $\alpha = -\infty$, $\beta = +\infty$.
2. Traverse the game tree in depth-first manner.
3. At each node:
 - **Maximizer:** update $\alpha = \max(\alpha, \text{value})$.
 - **Minimizer:** update $\beta = \min(\beta, \text{value})$.
 - If at any point $\alpha \geq \beta \rightarrow$ prune (ignore that branch).

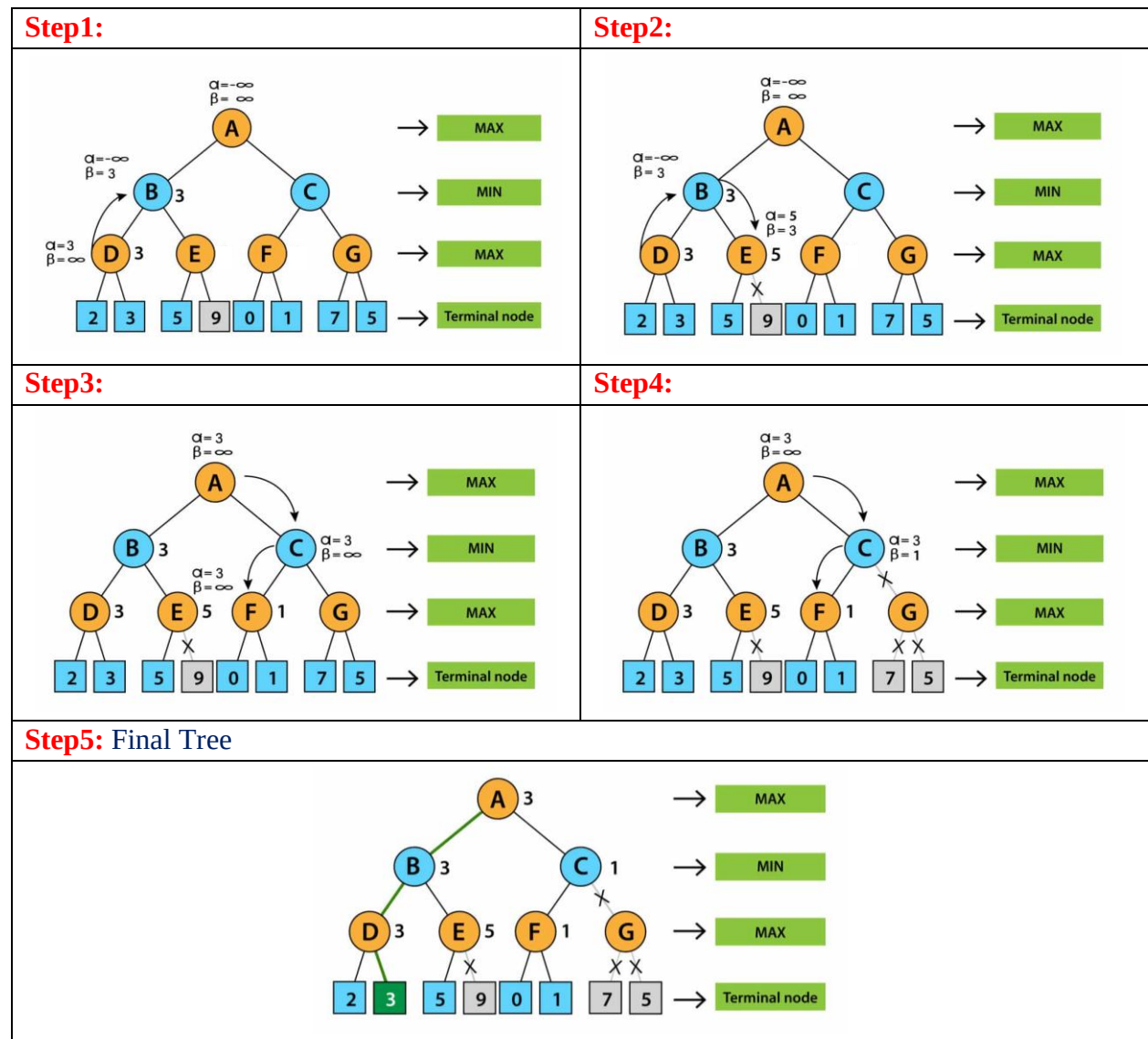
Properties

Property	Value
Completeness	Yes (same as Minimax)
Optimality	Yes (same result as Minimax)
Time	$O(b^{(d/2)})$ (much faster than Minimax)
Space	$O(bd)$ (same as DFS recursion)

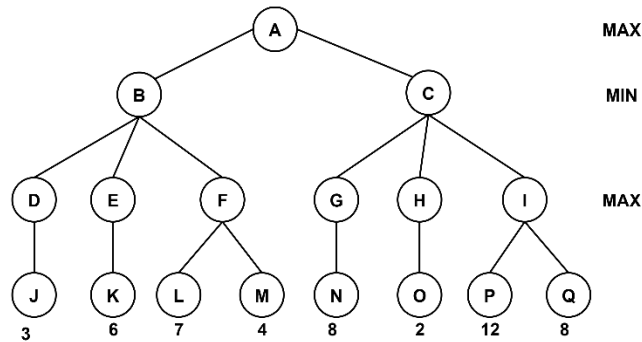
✂ **Problem1:** Apply the **alpha-beta pruning algorithm** to the game tree given below and **show the search tree** that would be built by this algorithm. Make sure that you show where the **alpha and beta cuts** are applied, and **which parts of the search tree are pruned** as a result.



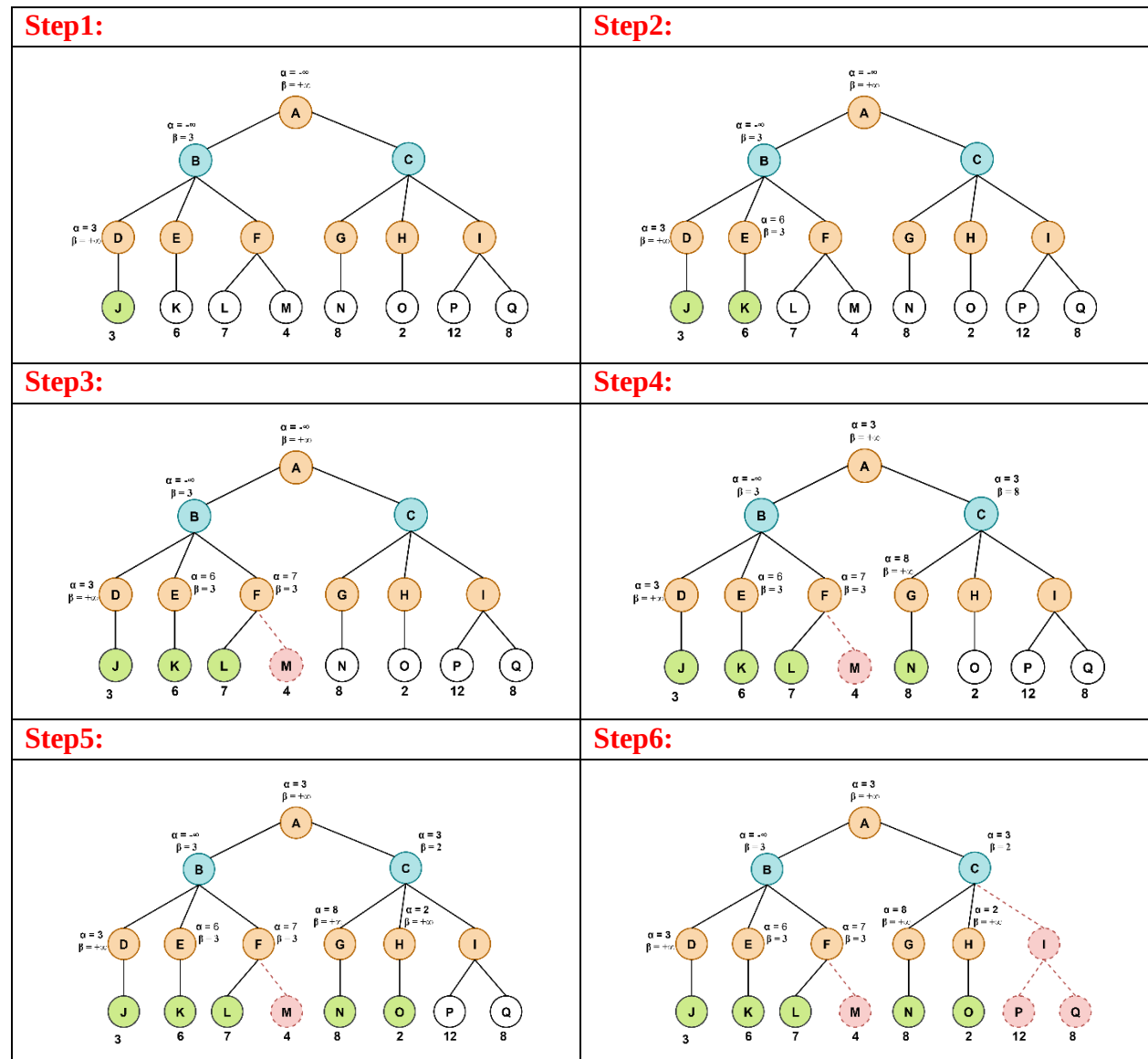
Solutions:



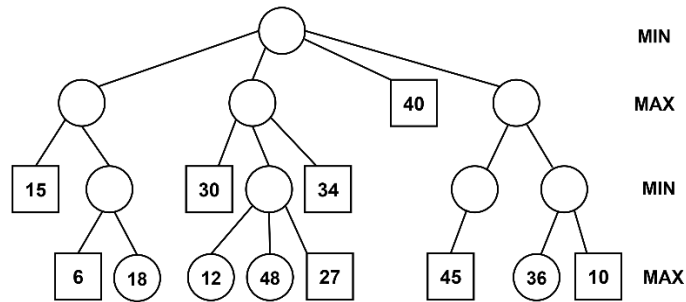
✂ **Problem2:** Apply the α - β pruning algorithm to the given minimax search tree. Clearly show each update of the α and β values at every step of the algorithm. Indicate exactly where the α and β cutoffs occur and specify which branches of the search tree are pruned as a result.



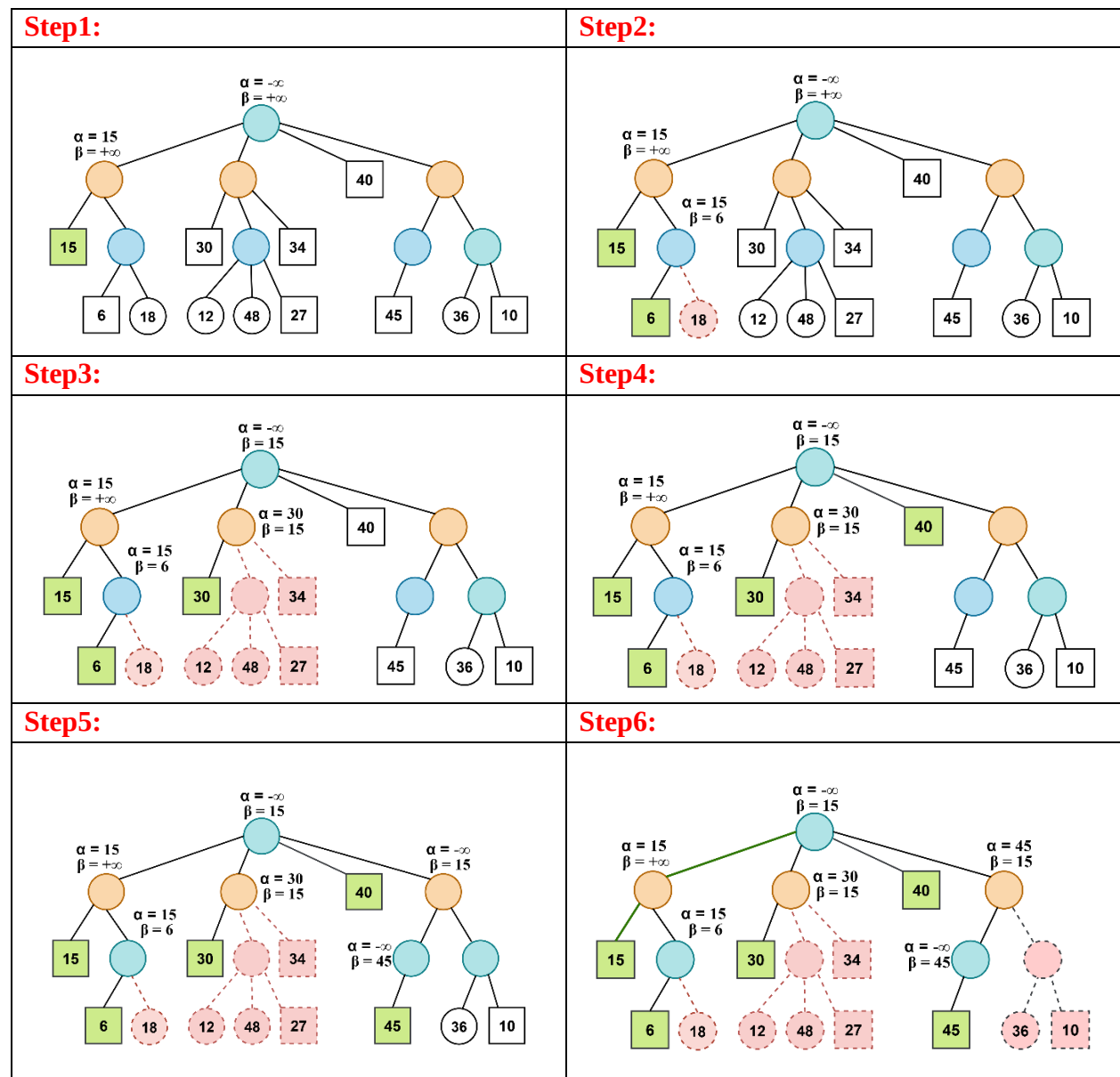
Solutions:



🔗 **Problem3:** Apply the α - β pruning algorithm to the given minimax search tree. Clearly show each update of the α and β values at every step of the algorithm. Indicate exactly where the α and β cutoffs occur and specify which branches of the search tree are pruned as a result.



Solutions:



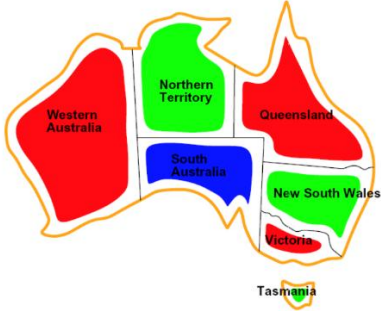
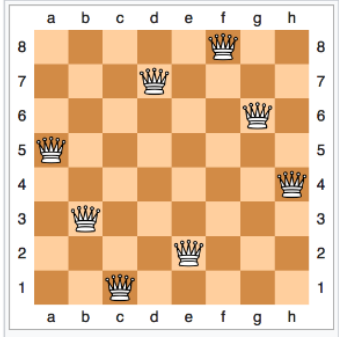
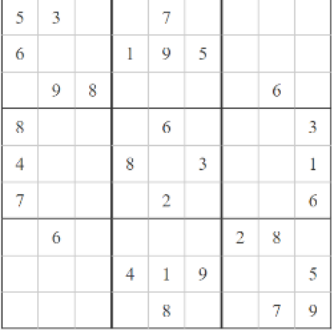
CONSTRAINT SATISFACTION PROBLEMS (CSPS)

Constraint Satisfaction Problems (CSPs)

- A **Constraint Satisfaction Problem (CSP)** is a problem defined by:
 1. **Variables** → a set of unknowns that need values.
 2. **Domains** → possible values each variable can take.
 3. **Constraints** → restrictions on values of variables.

Note: The goal is to **assign values to all variables** such that **all constraints are satisfied**.

Examples of CSPs

➤ Map Coloring Problem	➤ N-Queens Problem
	
• Variables: {WA, NT, Q, NSW, V, SA, T} • Domains: {Red, Green, Blue} • Constraints: Adjacent regions cannot have the same color.	• Variables: N queens (positions). • Domains: Board positions. • Constraints: No two queens attack each other.
➤ Sudoku Puzzle:	
	
○ Variables: Each empty cell. ○ Domains: {1–9}. ○ Constraints: No repetition in row, column, 3×3 box.	

Solving CSPs

a) Backtracking Search (DFS for CSPs)

- Assign values one variable at a time.
- If a variable violates a constraint, **backtrack**.
- Very inefficient without improvements.

b) Improving Backtracking Efficiency

- **Minimum Remaining Values (MRV)**
 - Choose the variable with the fewest legal values left.
- **Degree Heuristic**
 - Choose the variable involved in the most constraints.
- **Least Constraining Value (LCV)**
 - Try the value that rules out the fewest options for other variables.

c) Forward Checking

- After assigning a variable, eliminate inconsistent values from neighbors' domains.

d) Constraint Propagation (Arc Consistency)

- Use algorithms like **AC-3** to reduce domains by enforcing constraints before continuing the search.

Properties of CSPs

Feature	Value
Completeness	Yes (with backtracking + heuristics)
Optimality	Yes (if solution exists, will find it)
Time	Exponential in worst case (improved with heuristics)
Space	$O(n)$, depth-first style (linear in number of variables)

✂ **Problem1:** Consider the **Map Coloring Problem**, In this problem, you can color **each city** using one color from the following set of 3 colors, {**Red, Green, Blue**}. No two adjacent city has the same color.



- Represent this as constraint satisfaction problem (using **variables**, **domains** and **constraints**) and draw the **constraint graph**.
- Solve the problem by applying **Forward Checking**.

➔ **Solutions:**

(a)

Represent as CSP	Constraint Graph	
Variables: {WA, NT, Q, NSW, V, SA, T} Domains: {R, G, B}	<pre>graph LR; WA --- NT; WA --- SA; NT --- Q; NT --- SA; Q --- NSW; SA --- Q; SA --- NSW; SA --- V; NSW --- V; T</pre>	
Constraints		
WA≠NT		WA≠SA
NT≠Q	SA≠Q	SA≠NSW
SA≠V	Q≠NSW	NSW≠V

(b)

Forward Checking							
	WA	NT	Q	NSW	V	SA	T
Initial Domain	R G B	R G B	R G B	R G B	R G B	R G B	R G B
WA = R	R	G B	R G B	R G B	R G B	G B	R G B
Q = R	R	G B	R	G B	R G B	G B	R G B
V = R	R	G B	R	G B	R	G B	R G B
T = G	R	G B	R	G B	R	G B	G
NT = G	R	G	R	G B	R	B	G
NSW = G	R	G	R	G	R	B	G
SA = B	R	G	R	G	R	B	G

✂ **Problem2:** Consider the **Map Coloring Problem**, In this problem, you can color **each city** using one color from the following set of 3 colors, {**Red, Green, Blue**}. No two adjacent city has the same color.



- Represent this as constraint satisfaction problem (using **variables**, **domains** and **constraints**) and draw the **constraint graph**.
- Solve the problem by applying **back tracking search** using **minimum remaining value** heuristic.

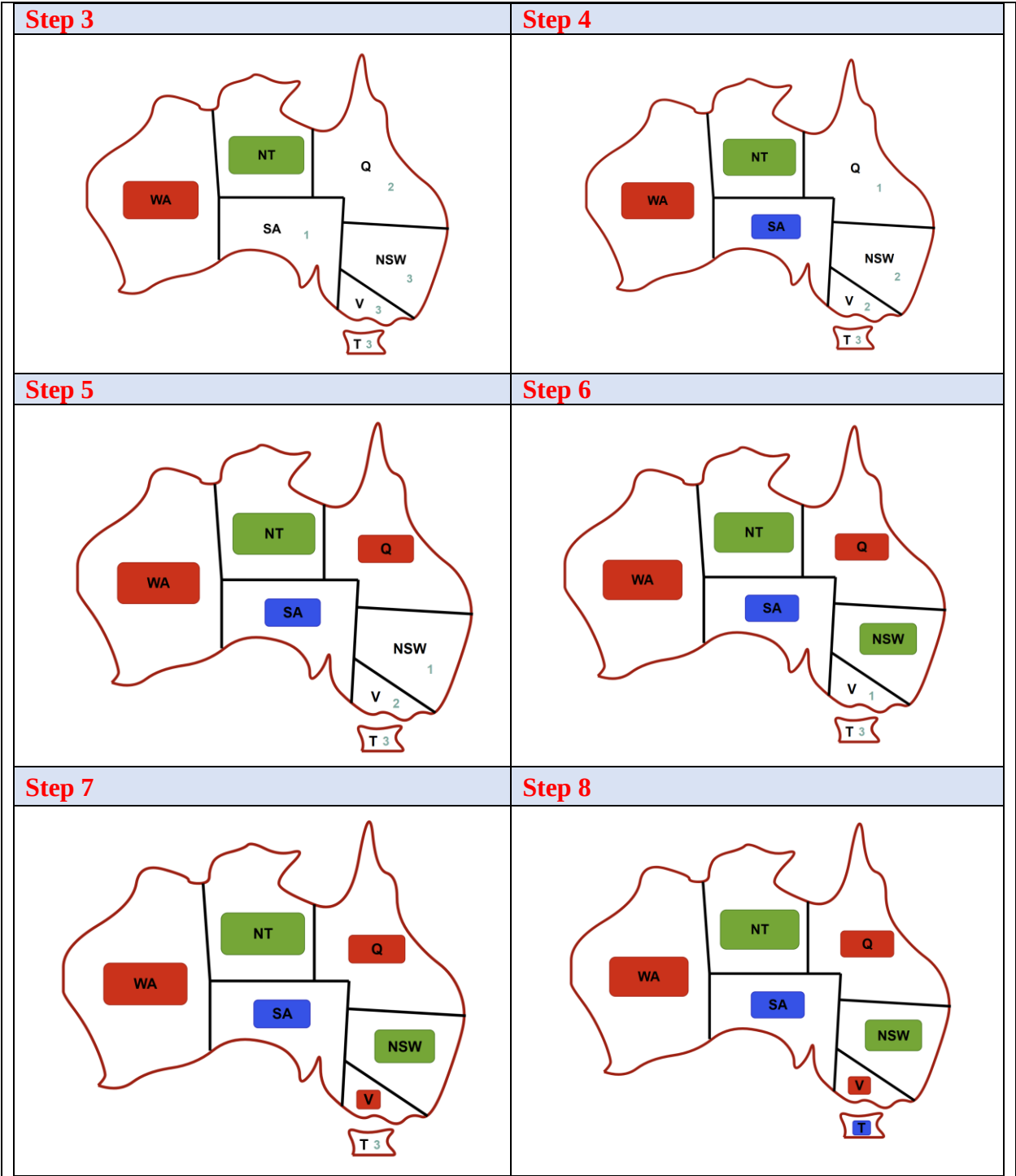
➔ **Solutions:**

(a)

Represent as CSP	Constraint Graph												
Variables: {WA, NT, Q, NSW, V, SA, T} Domains: {R, G, B} <table><tr><th colspan="3">Constraints</th></tr><tr><td>WA≠NT</td><td>WA≠SA</td><td>NT≠SA</td></tr><tr><td>NT≠Q</td><td>SA≠Q</td><td>SA≠NSW</td></tr><tr><td>SA≠V</td><td>Q≠NSW</td><td>NSW≠V</td></tr></table>	Constraints			WA≠NT	WA≠SA	NT≠SA	NT≠Q	SA≠Q	SA≠NSW	SA≠V	Q≠NSW	NSW≠V	<pre>graph LR; WA --- NT; WA --- SA; NT --- SA; NT --- Q; SA --- Q; SA --- NSW; Q --- NSW; NSW --- V; T --- None</pre>
Constraints													
WA≠NT	WA≠SA	NT≠SA											
NT≠Q	SA≠Q	SA≠NSW											
SA≠V	Q≠NSW	NSW≠V											

(b)

back tracking search using minimum remaining value (MRV)	
Step 1	Step 2



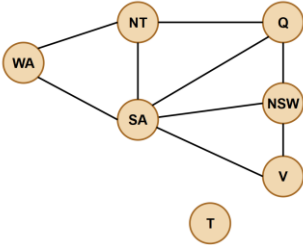
✂ **Problem3:** Consider the **Map Coloring Problem**, In this problem, you can color **each city** using one color from the following set of 3 colors, {**Red, Green, Blue**}. No two adjacent city has the same color.



- Represent this as constraint satisfaction problem (using **variables**, **domains** and **constraints**) and draw the **constraint graph**.
- Solve the problem by applying **back tracking search** using **Degree Heuristic / most constraints variable**

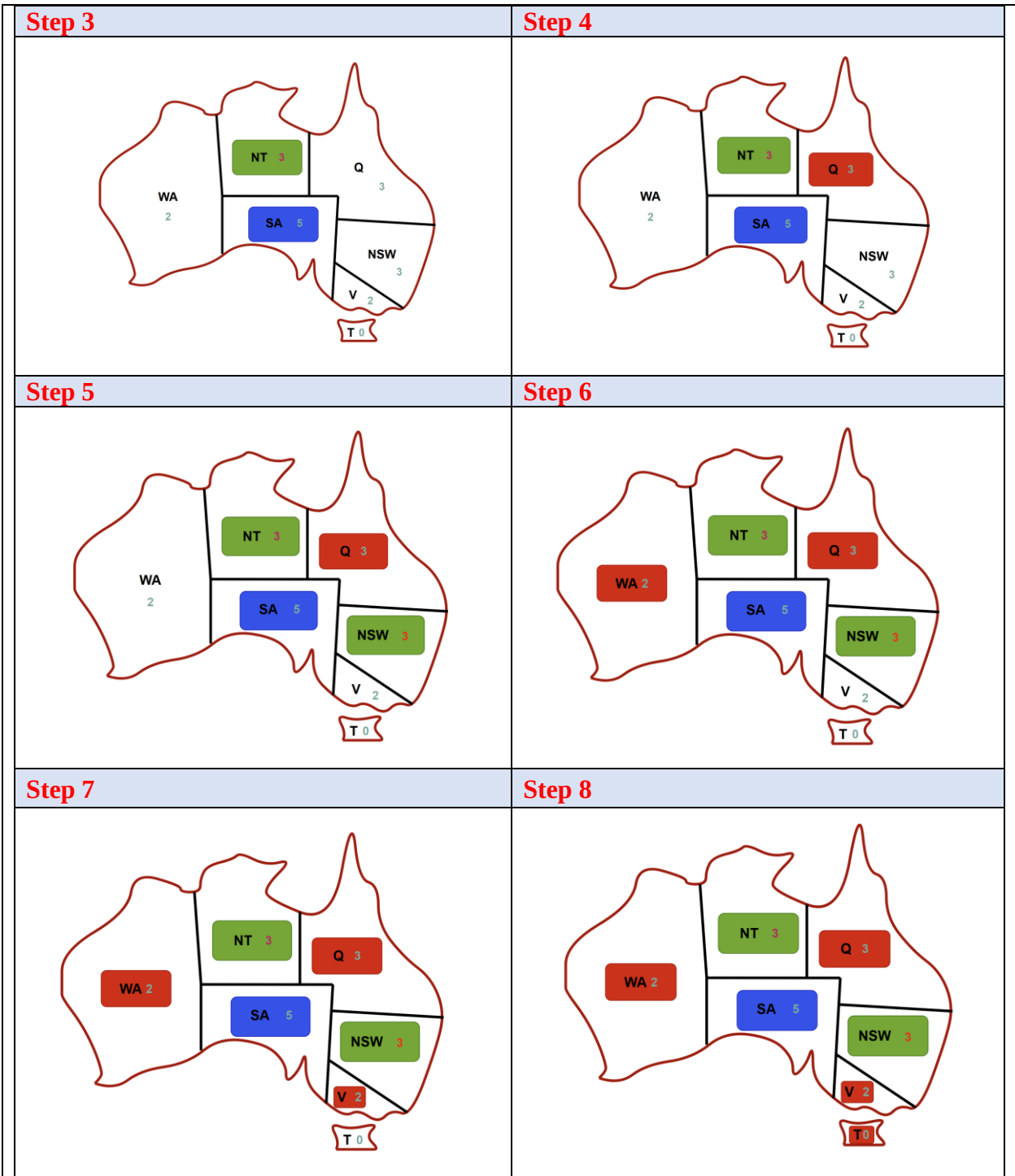
➔ **Solutions:**

(a)

Represent as CSP	Constraint Graph												
Variables: {WA, NT, Q, NSW, V, SA, T} Domains: {R, G, B}													
<table><tr><th colspan="3">Constraints</th></tr><tr><td>WA≠NT</td><td>WA≠SA</td><td>NT≠SA</td></tr><tr><td>NT≠Q</td><td>SA≠Q</td><td>SA≠NSW</td></tr><tr><td>SA≠V</td><td>Q≠NSW</td><td>NSW≠V</td></tr></table>		Constraints			WA≠NT	WA≠SA	NT≠SA	NT≠Q	SA≠Q	SA≠NSW	SA≠V	Q≠NSW	NSW≠V
Constraints													
WA≠NT	WA≠SA	NT≠SA											
NT≠Q	SA≠Q	SA≠NSW											
SA≠V	Q≠NSW	NSW≠V											

(b)

back tracking search using Degree Heuristic / most constraints variable	
Step 1	Step 2



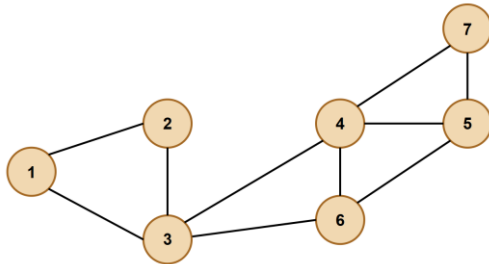
✂ **Problem4:** Consider the **Block Puzzle Problem**, In this problem, you can color **each Block** using one color from the following set of 3 colors, {**Red, Green, Blue**}. No two adjacent Block has the same color.

1	3	4		
2		6	5	7

- Represent this as constraint satisfaction problem (using **variables**, **domains** and **constraints**) and draw the **constraint graph**.
- Solve the problem by applying **back tracking search** using **minimum remaining value** heuristic.

→ **Solutions:**

(a)

Represent as CSP	Constraint Graph															
Variables: {1, 2, 3, 4, 5, 6, 7} Domains: {R, G, B} <table><tr><th colspan="3">Constraints</th></tr><tr><td>1 ≠ 2</td><td>1 ≠ 3</td><td>2 ≠ 3</td></tr><tr><td>3 ≠ 4</td><td>3 ≠ 6</td><td>4 ≠ 6</td></tr><tr><td>4 ≠ 5</td><td>4 ≠ 7</td><td>5 ≠ 6</td></tr><tr><td>5 ≠ 7</td><td></td><td></td></tr></table>	Constraints			1 ≠ 2	1 ≠ 3	2 ≠ 3	3 ≠ 4	3 ≠ 6	4 ≠ 6	4 ≠ 5	4 ≠ 7	5 ≠ 6	5 ≠ 7			
Constraints																
1 ≠ 2	1 ≠ 3	2 ≠ 3														
3 ≠ 4	3 ≠ 6	4 ≠ 6														
4 ≠ 5	4 ≠ 7	5 ≠ 6														
5 ≠ 7																

(b)

back tracking search using minimum remaining value (MRV)																												
Step 1					Step 2																							
<table><tr><td>1₃</td><td rowspan="2">3₃</td><td colspan="3">4₃</td></tr><tr><td>2₃</td><td>6₃</td><td>5₃</td><td>7₃</td></tr></table>					1 ₃	3 ₃	4 ₃			2 ₃	6 ₃	5 ₃	7 ₃	<table><tr><td>1</td><td rowspan="2">3₂</td><td colspan="3">4₃</td></tr><tr><td>2₂</td><td>6₃</td><td>5₃</td><td>7₃</td></tr></table>					1	3 ₂	4 ₃			2 ₂	6 ₃	5 ₃	7 ₃	
1 ₃	3 ₃	4 ₃																										
2 ₃		6 ₃	5 ₃	7 ₃																								
1	3 ₂	4 ₃																										
2 ₂		6 ₃	5 ₃	7 ₃																								
Step 3					Step 4																							
<table><tr><td>1</td><td rowspan="2">3₁</td><td colspan="3">4₃</td></tr><tr><td>2</td><td>6₃</td><td>5₃</td><td>7₃</td></tr></table>					1	3 ₁	4 ₃			2	6 ₃	5 ₃	7 ₃	<table><tr><td>1</td><td>3</td><td colspan="3">4₂</td></tr><tr><td>2</td><td></td><td>6₂</td><td>5₃</td><td>7₃</td></tr></table>					1	3	4 ₂			2		6 ₂	5 ₃	7 ₃
1	3 ₁	4 ₃																										
2		6 ₃	5 ₃	7 ₃																								
1	3	4 ₂																										
2		6 ₂	5 ₃	7 ₃																								

Step 5	Step 6																		
<table><tr><td>1</td><td rowspan="2">3</td><td colspan="3">4</td></tr><tr><td>2</td><td>6₁</td><td>5₂</td><td>7₂</td></tr></table>	1	3	4			2	6 ₁	5 ₂	7 ₂	<table><tr><td>1</td><td rowspan="2">3</td><td colspan="3">4</td></tr><tr><td>2</td><td>6</td><td>5₁</td><td>7₂</td></tr></table>	1	3	4			2	6	5 ₁	7 ₂
1	3		4																
2		6 ₁	5 ₂	7 ₂															
1	3	4																	
2		6	5 ₁	7 ₂															
Step 7	Step 8																		
<table><tr><td>1</td><td rowspan="2">3</td><td colspan="3">4</td></tr><tr><td>2</td><td>6</td><td>5</td><td>7₁</td></tr></table>	1	3	4			2	6	5	7 ₁	<table><tr><td>1</td><td rowspan="2">3</td><td colspan="3">4</td></tr><tr><td>2</td><td>6</td><td>5</td><td>7</td></tr></table>	1	3	4			2	6	5	7
1	3		4																
2		6	5	7 ₁															
1	3	4																	
2		6	5	7															

✂ **Problem5:** Identify the values of **A, B, C** using **Forward Checking**: $A = \{1,2,3\}$, $B = \{1,2,3\}$, $C = \{1,2,3\}$ and **Constraints**: $A = B$, $B > C$, also find all possible consistent assignments.

Solutions:

Forward Checking			
	A	B	C
Initial Domain	1 2 3	1 2 3	1 2 3
$A = 2$	2	2	1 2 3
$B = 2$	2	2	1
$C = 1$	2	2	1
or			
$A = 3$	3	3	1 2 3
$B = 3$	3	3	1 2
$C = 1$	3	3	1
or			
$A = 3$	3	3	1 2 3
$B = 3$	3	3	1 2
$C = 1$	3	3	2

Final solutions (all consistent assignments)

- (A=2, B=2, C=1)
- (A=3, B=3, C=1)
- (A=3, B=3, C=2)

✂ **Problem6:** Identify the values of **A, B, C** using **Forward Checking**: $A = \{1,2,3\}$, $B = \{1,2,3\}$, $C = \{1,2,3\}$ and **Constraints**: $A \neq B$, $B \neq C$, $A < C$

Solutions:

Forward Checking			
	A	B	C
Initial Domain	1 2 3	1 2 3	1 2 3
A = 1	1	2 3	2 3
B = 2	1	2	3
C = 3	1	2	3

✂ **Problem7:** Identify the values of **A, B, C** using **Forward Checking**: $A = \{0,1,2\}$, $B = \{0,2,3\}$, $C = \{0,1,3\}$ and **Constraints**: $A \neq B$, $B \neq C$, $A < C$

Solutions:

Forward Checking			
	A	B	C
Initial Domain	0 1 2	0 2 3	0 1 3
A = 2	2	0 3	0 1
B = 0	2	0	1
C = 1	2	0	1

BAYESIAN NETWORK

Bayesian Networks (Belief Networks)

⇒ A **Bayesian Network (BN)** is a **probabilistic graphical model** that represents a set of variables and their conditional dependencies using a **Directed Acyclic Graph (DAG)**.

Note: It provides a compact way to represent **joint probability distributions (JPDs)**.

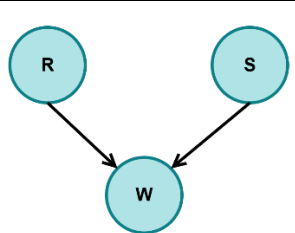
Components of a Bayesian Network

- **Nodes** → represent **random variables** (e.g., Rain, Traffic, Sprinkler).
- **Edges (arcs)** → represent **dependencies** between variables.
 - A directed edge from $A \rightarrow B$ means **A influences B**.
- **Conditional Probability Tables (CPTs)** → specify the probability of a node given its parents.

Joint Probability in Bayesian Networks

⇒ For variables $X_1, X_2, \dots, X_n$:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

Example: Rain, Sprinkler, and Wet Grass	
Variables:	Graph:
• R = Rain • S = Sprinkler • W = Wet Grass	 <pre>graph TD; R((R)) --> W((W)); S((S)) --> W((W));</pre>
Dependencies:	Joint probability:
• Rain affects Wet Grass • Sprinkler affects Wet Grass	$P(R, S, W) = P(W \mid R, S) \times P(R) \times P(S)$

Examples of 3-way Bayesian Networks	